

WeCare: Online Catering Reservation System



Submitted by	Muhammad Ali Sadam Shiekh Usama Mohsin
Roll. No	0120-BSCS-20 0102-BSCS-20 0099-R-BSCS-20
Session	2020 - 2024
Supervised by	Dr. Awais Qasim Assistant Professor

**BS(HONS)
IN
COMPUTER SCIENCE**

**DEPARTMENT OF COMPUTER SCIENCE
GC UNIVERSITY LAHORE**

WeCare: Online Catering Reservation System

**Submitted to GC University Lahore in partial fulfillment
of the requirements for the award of degree of**

BS(HONS) IN COMPUTER SCIENCE

Submitted by	Muhammad Ali Sadam Shiekh Usama Mohsin
Roll. No	0120-BSCS-20 0102-BSCS-20 0099-R-BSCS-20
Session	2020 - 2024
Supervised by	Dr. Awais Qasim Assistant Professor

**DEPARTMENT OF COMPUTER SCIENCE
GC UNIVERSITY LAHORE**

Declaration

We, Muhammad Ali, Usama Mohsin, and Sadam Shiekh current students of the degree **BS(Hons) in Computer Science** as a session of **2020-2024**, preent that the content printed in this thesis named as, WeCare: Online Catering Reservation System is our personal work. It has not been printed, published and submitted prior, in any form in any University, Research Institution etc in Pakistan or abroad.

Date: _____

Signatures of Deponent

Research Completion Certificate

It is certified that the research work contained in this thesis titled WeCare: Online Catering Reservation System has been carried out by **Muhammad Ali, Usama Mohsin, and Sadam Shiekh** Roll. No **0120-BSCS-20. 0099-R-BSCS-20, and 0102-BSCS-20** under my supervision.

Dr. Awais Qasim
Assistant Professor

Date: _____

Submitted Through

Prof. Dr. M Waqas Anwar
Chairperson
Department of Computer Science
GC University Lahore

Controller of Examination
GC University Lahore

Acknowledgements

We extend our deepest gratitude to Almighty **Allah** for His countless blessings. We also offer our heartfelt thanks to the Holy Prophet Muhammad (S.A.W), the epitome of ideal and virtue. This dissertation was completed under the great supervision and guidance of **Dr. Awais Qasim**, Assistant Professor at GC University, Lahore. His unwavering moral support, insightful guidance, constructive criticism, and valuable advice have been instrumental throughout our study.

We also wish to express our sincere appreciation to our beloved parents, who have been our greatest source of confidence and strength. Our heartfelt thanks go to our friends as well, for their continuous encouragement and prayers for our success.

Dedication

Our Documentation work is devoted to my beloved family and our worthy supervisor Dr. Awais Qasim who heartened and assisted us to complete our research in the domain of WeCare Web-App associated to computer science.

Abstract

The catering industry faces noteworthy challenges in managing orders and inventory. Traditional methods of placing and tracking catering orders are often inefficient that leads to delays and miscommunication. To address these issues, we propose the development of a web-based application named *WeCare: Online Catering Reservation System*. This project goals to transform the catering experience by providing an instinctive and well-organized platform for event organizers to place and manage their orders online. Users can peruse a pre-defined menu, add items to a cart, fill in delivery details, and make payments through integrated payment gateways such as direct through card. The system offers real-time order tracking, ensuring that users are always well-versed about the status of their orders. Also, the platform allows caterers to manage orders, and track payments. By streamlining the order process, WeCare goals to improve working efficiency and improve user satisfaction in the industry.

Contents

Declaration	i
Research Completion Certificate	ii
Acknowledgements	iii
Dedication	iv
Abstract	v
Contents	vi
List of Tables	ix
List of Figures	x
1 Introduction	1
1.1 Introduction	1
1.2 Background	2
1.3 Problem Statement	2
1.4 Scope of the Project	2
1.5 Aims of the project	3
1.6 Literature Review	3
2 Requirement Specification	4
2.1 Tool Used	6
2.1.1 Technologies Used	7
2.1.1.1 React	7
2.1.1.2 Node.js	7
2.1.1.3 Express.js	7
2.1.1.4 MongoDB	7

3	Design of the Project	9
3.1	Methodology	9
3.2	Architecture Overview	11
3.2.1	Entity-Relationship Diagram	11
3.2.2	UseCase Diagram	12
3.2.3	Use Cases	13
3.2.3.1	User Registration	13
3.2.3.2	User Registration	13
3.2.3.3	Sign Out	14
3.2.3.4	View Menu	14
3.2.3.5	Add products to cart	15
3.2.3.6	Remove Product Items from Cart	15
3.2.3.7	Update products Item Quantity	16
3.2.3.8	Search desired products	16
3.2.3.9	Process Payment	17
3.2.3.10	Manage User Profile	17
3.2.3.11	View Order History	18
3.2.3.12	Admin Sign In	18
3.2.3.13	Admin Dashboard Access	19
3.2.3.14	Add new product Item	19
3.2.4	Class Diagram	20
3.3	Design Description	21
3.3.1	Model	21
3.3.2	View	21
3.3.3	Controller	21
4	Implementation and Evaluation	22
4.1	Development Stages	22
4.1.1	Strategy	22
4.2	System Integration	23
4.2.1	User Interface	23
4.2.1.1	Home Screen	24
4.2.1.2	View Product Screen	25
4.2.1.3	Cart Screen	26
4.2.1.4	Product Search	27
4.2.1.5	Sign Up	28
4.2.1.6	Sign In	29
4.2.1.7	Shipping Address	30
4.2.1.8	Payment Method Screen	31
4.2.1.9	Order Screen	32
4.2.1.10	Order History Screen	33
4.2.1.11	Order Details Screen	34
4.2.1.12	Payment Details Screen	35
4.2.1.13	User Profile Screen	36

4.2.1.14	Customer Options	37
4.2.1.15	Admin Options	38
4.2.1.16	Add New Product Screen	39
4.2.1.17	Admin Order Management Screen	40
4.2.1.18	Admin Product Management Screen	41
4.2.1.19	Admin User Management Screen	42
4.2.1.20	Admin Dashboard Screen	43
4.2.1.21	Edit Product Screen	44
4.3	Evaluation	45
4.4	Unit Testing	45
4.5	Functional Testing	45
4.6	Test Cases	46
4.6.1	User Registration	46
4.6.2	User Sign In	47
4.6.3	Sign Out	47
4.6.4	Add Product to Cart	48
4.6.5	Remove Product from Cart	48
4.6.6	Search Products	49
4.6.7	Payment Method	49
4.6.8	Manage User Profile	50
4.6.9	Admin Sign In Access	50
4.6.10	View Menu Display	51
4.6.11	Admin Dashboard Management	51
5	Conclusion & Future Work	52
5.1	Conclusion	52
5.2	Future Work	53

Bibliography	54
---------------------	-----------

List of Tables

List of Figures

3.1	ERD Diagram	11
3.2	UseCase Diagram	12
3.3	Class Diagram	20
4.1	Home Screen	24
4.2	View Product Screen	25
4.3	Cart Screen	26
4.4	Product Search	27
4.5	Sign Up	28
4.6	Sign In	29
4.7	Shipping Address	30
4.8	Payment Method Screen	31
4.9	Order Screen	32
4.10	Order History Screen	33
4.11	Order Details Screen	34
4.12	Payment Details Screen	35
4.13	User Profile Screen	36
4.14	Customer Options	37
4.15	Admin Options	38
4.16	Add New Product Screen	39
4.17	Admin Order Management Screen	40
4.18	Admin Product Management Screen	41
4.19	Admin User Management Screen	42
4.20	Admin dashboard Screen	43
4.21	Edit Product Screen	44

Chapter 1

Introduction

1.1 Introduction

The WeCare: Online Catering Reservation System project presents an advanced web-based platform designed to modernize the industry. WeCare functions as a well-organized and user-central solution. It differentiates itself from outmoded methods of managing catering services. This platform links the gap between event organizers and catering service providers.

The main objective of WeCare is to offer a user-friendly and complete online environment. By doing so, it streamlines the process of placing and managing catering orders and makes it more open for event organizers to connect with catering services. This cooperation between technology and catering services promises to bring about a significant revolution in how catering orders are managed and delivered. The dominant pillars of WeCare are Accessibility, Convenience, and Efficiency. It aims to transform the catering industry by seamlessly integrating technology into its services.

1.2 Background

The WeCare: Online Catering Reservation System project is meant at streamlining the catering industry. In comeback to the dynamic requirements of event management, this creativity presents WeCare, a complete digital platform controlled to change the way event organizers and catering service providers interact.

WeCare endeavors to transform the catering industry through technology that makes the process of placing and managing catering orders more open, suitable, and well-organized. Its purpose is to bridge the gap between event organizers and catering service providers that fosters smoother and more effective events.

1.3 Problem Statement

The traditional methods of managing catering orders in the event industry often depend on manual procedures that leads to incompetence and working challenges. For example, event organizers typically involve in inefficient communications through phone or browser to place and manage catering orders, which can result in mistakes and delays. Also, catering service providers may face problems in precisely tracking inventory and managing orders without a unified system that leads to potential errors and customer disappointment. These challenges highlight the perilous need for a digital solution that rationalizes the catering order process, and ensures well-organized management of resources.

1.4 Scope of the Project

Effective Order Placement: The WeCare project purposes to rationalize the process of placing catering orders for events. It offers event organizers with an instinctive platform to browse a pre-defined menu, select dishes, specify quantities, and manage orders impeccably.

Caterer Management: WeCare comprises features for caterers to manage incoming orders, track payments, and monitor inventory efficiently through a dedicated admin dashboard. This functionality enhances working efficiency and customer satisfaction.

User-friendly Interface: The platform will present a user-friendly interface that improves convenience and convenience for event organizers and simplifies the catering order process.

Integration with Payment Gateways: WeCare integrates with payment gateways that will facilitate secure and suitable online payments for catering services such as via Mobile and Bank Card transfer.

1.5 Aims of the project

The aim of the WeCare: Online Catering Reservation System project is to present a comprehensive web-based platform. It will streamline the process of placing and managing catering orders for events. This project seeks to improve suitability, competence, and accessibility for event organizers through a all-in-one online system.

1.6 Literature Review

The catering industry has usually depended on manual processes for order placement and management that leads to inadequacies and communication issues. Recent studies highlight the need for digital solutions to modernize these operations. For instance, research highlights the benefits of integrating online platforms which significantly enhances working efficiency and customer satisfaction[1]. Also, the implementation of secure online payment systems has been shown to improve transaction reliability and user accessibility that strengthens the shift towards digital transformation in the catering sector.

Chapter 2

Requirement Specification

Functional Requirements

1. User Registration and Sign In

- Users (event organizers and caterers) must be able to register and log in to the system.
- The system should verify user credentials and provide access based on user roles.

2. Menu Browsing and Selection

- Event organizers should be able to browse a predefined menu of dishes.
- Users must be able to add selected dishes to their cart and specify quantities.

3. Order Placement

- Users should be able to place orders by providing necessary details (e.g., event date, location).
- The system should allow users to review and confirm their orders.

4. Online Payment

- The platform should integrate secure online payment gateways (e.g., Mobile, bank accounts).
- Users must be able to make payments online.

5. Real-time Order Tracking

- Users should receive updates on the status of their orders whether confirmed or not.
- The system should notify users about order approval and delivery date.

6. Admin Panel

- Administrators should have access to a panel for managing users, orders, and inventory.
- The admin panel must provide the option to add new products or dishes.

Non-Functional Requirements

1. Usability

- The application should have an instinctive and user-friendly interface.
- Navigation should be frank for users of all technical proficiency levels.

2. Performance

- The system should handle multiple concurrent users without performance degradation.
- Real-time updates and transactions must be handled within acceptable time limits.

3. Security

- The application must ensure the security of user data and transactions.
- The system should comply with pertinent data protection rules and standards.

4. Reliability

- The platform should be consistent and available with negligible down-time.
- Data integrity must be kept even in the event of system failures.

5. Scalability

- The system should be scalable to handle an upward number of users and transactions.
- It should support future improvements and additional features.

6. Compatibility

- The application must be well-matched with numerous web browsers and devices.
- It should ensure reliable functionality across diverse platforms.

2.1 Tool Used

- React
- MongoDB
- Node.js
- Express.js
- Visual Studio
- Postman & Thunder Client

2.1.1 Technologies Used

2.1.1.1 React

React helps as the chief technology for developing the web application component of the WeCare system. React allows the creation of dynamic and interactive user interfaces known for its component-based architecture and effectual rendering. It allows for code reusability and well-organized management of the application's state that contributes to the development of a responsive and user-friendly web application.

2.1.1.2 Node.js

Node.js is used for developing backend services in the WeCare project. Node.js offers non-blocking, event-driven functionality as a server-side JavaScript runtime. It also boosts the competence of backend operations. The noteworthy fact is that it is lightweight and scalable features that makes it perfect for the demands of a modern web application. It offers a strong foundation for backend development.

2.1.1.3 Express.js

Express.js is used in the WeCare project to build the backend services. Express.js offers robust features for web applications as a negligible and flexible Node.js web application framework. Its uncomplicatedness and performance safeguard rapid development and effectual handling of HTTP requests and responses that makes it an first-rate choice for creating RESTful APIs and handling the backend logic of WeCare.

2.1.1.4 MongoDB

MongoDB helps as the database solution for the WeCare project. It offers a flexible and scalable tactic to managing data. MongoDB excels in handling varied

and unstructured data types as a NoSQL database, which is perfect for the dynamic requirements of modern applications like WeCare. Its document-oriented model allows for unified integration with Node.js and Express.js that ensures well-organized data storage and retrieval. MongoDB's support for high accessibility, horizontal scaling, and flexible schema design aligns flawlessly with WeCare's requirements. It also enables strong data management and scalability as the project grows.

Chapter 3

Design of the Project

3.1 Methodology

In the WeCare project, we will use the iterative model. This approach allows for continuous refinement and enhancement of features through systematic iterative cycles. Each step is discussed below:

- **Planning and Requirements:**

- Describe the project scope, goals, and objectives.
- Gather and evaluate user requirements.
- Create a roadmap outlining iterative phases and milestones.

- **Analysis and Design:**

- Conduct comprehensive analysis of gathered requirements.
- Design solutions and system components.
- Develop prototypes or mockups to confirm design concepts and functionalities.

- **Implementation:**

- Start development based on the approved designs and prototypes.

-
- Incrementally build features and functionalities in iterative cycles.
 - Ensure code quality and obedience to coding standards.
 - **Testing:**
 - Perform unit testing to authenticate individual components.
 - Conduct integration testing to safeguard modules work together flawlessly.
 - Perform system testing to validate functionality and performance.
 - **Evaluation:**
 - Review and assess each iteration against predefined goals and acceptance criteria.
 - Gather feedback from end-users.
 - Detect areas for improvement and rank adjustments for subsequent iterations.

3.2 Architecture Overview

3.2.1 Entity-Relationship Diagram

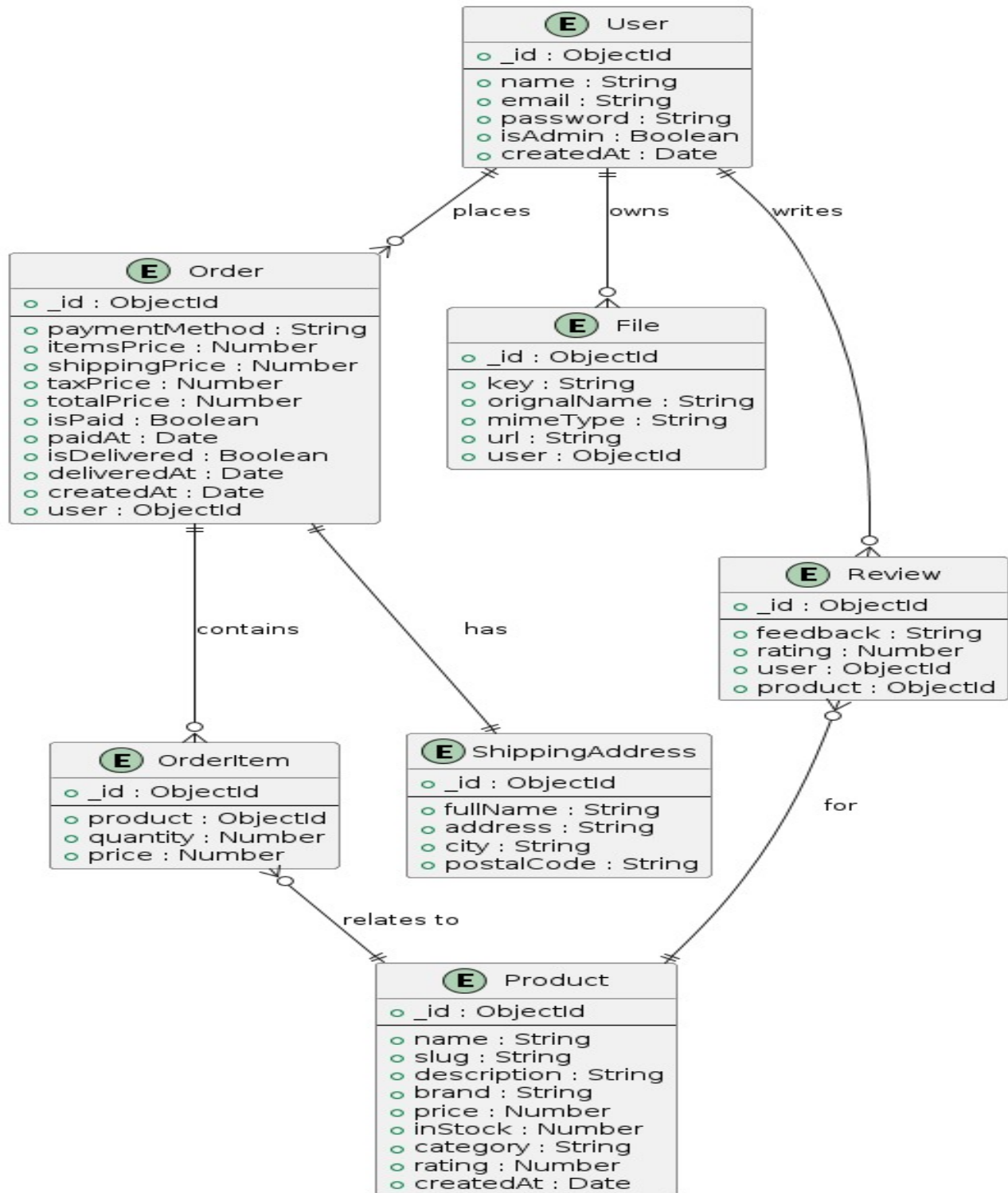


FIGURE 3.1: ERD Diagram

3.2.2 UseCase Diagram

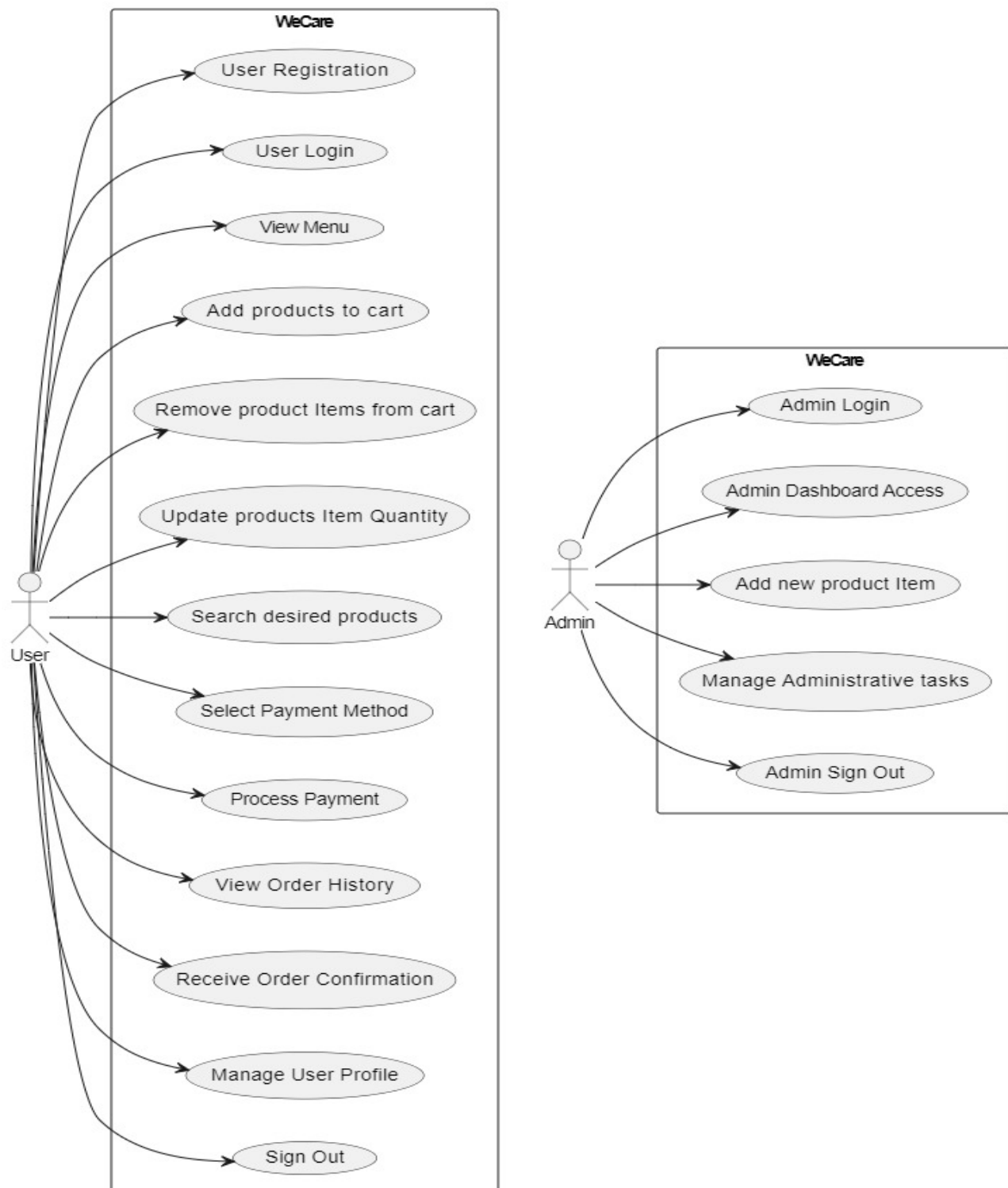


FIGURE 3.2: UseCase Diagram

3.2.3 Use Cases

In this section, we will see all the uses cases as well as their detailed description of the WeCare (Managing Catering Orders for Events) web application.

3.2.3.1 User Registration

Name	User Registration
Actors	End User
Description	To make payments, end user must be registered first.
Precondition	Actors open Website. They will have permission to see menu but get registered to make payments.
Scenario	1) User enters email and password. 2) Click on the register button. 3) Actors will be registered
Post Condition	User is registered and have permission to make payments for the orders.

3.2.3.2 User Registration

Name	User Sign In
Actors	All User
Description	User must sign in to access certain services, such as making payments.
Precondition	User opens the website and navigates to the sign in page
Scenario	1) User enters email and password. 2) Clicks on the Sign In button. 3) If the credentials are correct, the user is logged in successfully. 4) If the credentials are incorrect, an error message is displayed indicating invalid email or password.
Post Condition	User is Signed in and redirected to the home page.
Exception	If the credentials are incorrect, Sign In will not succeed.

3.2.3.3 Sign Out

Name	Sign Out
Actors	User
Description	User Signs out from the system.
Precondition	User must be Signed in.
Scenario	1) User navigates to the profile or settings page. 2) User clicks on the "Sign Out" button. 3) User will sign-out from the system.
Post Condition	User session is terminated, and user is logged out from the system.
Exception	If the user is not logged in, no action is taken.

3.2.3.4 View Menu

Name	View Menu
Actors	All Users
Description	Users can view the complete menu without logging in or registering.
Precondition	User opens the website.
Scenario	1) User navigates to the home page. 2) The system displays the complete menu with all available products.
Post Condition	The user can see the complete menu and select items to add to the cart.

3.2.3.5 Add products to cart

Name	Add products to cart
Actors	All Users
Description	Users can add products to their shopping cart for future purchase.
Precondition	User views the menu and selects products they wish to purchase.
Scenario	1) User navigates to the home page. 2) User selects a product and clicks the "Add to Cart" button. 3) The system adds the selected product to the user's cart and updates the cart icon with the number of items. 4) User can continue browsing and adding more products to the cart.
Post Condition	The selected products are added to the user's cart, and the cart is updated.

3.2.3.6 Remove Product Items from Cart

Name	Remove Product Items from Cart
Actors	End User
Description	User removes selected items if they need.
Precondition	User must have items in the cart.
Scenario	1) User navigates to the cart page. 2) User selects the items to remove. 3) User clicks on the delete button. 4) Items will be removed
Post Condition	Selected items are removed from the cart.
Exception	If the cart is empty, no items can be removed.

3.2.3.7 Update products Item Quantity

Name	Update products Item Quantity
Actors	User
Description	User increases or decreases the quantity of a product in their cart.
Precondition	User must be signed in and have added items to their cart.
Scenario	1) User navigates to the cart page. 2) User points out the product for which they want to update the quantity. 3) User adjusts the quantity by either increasing or decreasing the count.
Post Condition	Quantity of the selected product in the cart is updated accordingly.
Exception	If the an error occurs during the update process, the system displays an error message and does not update the quantity.

3.2.3.8 Search desired products

Name	Search desired products
Actors	User
Description	User searches for specific products using keywords or phrases.
Precondition	User must be on the homepage or a dedicated search page.
Scenario	1) User enters a search query into the search bar. 2) System displays relevant product suggestions as the user types. 3) User selects a specific product from the search results or refines their search criteria. 4) System displays matching products based on the search query.
Post Condition	User sees a list of products matching their search criteria.
Exception	If there are no matching products found for the search query, the system displays a message indicating no results found.

3.2.3.9 Process Payment

Name	Process Payment
Actors	User, Admin
Description	User initiates the payment process for their order using selected payment methods such as via Mobile or bank transfer.
Precondition	User has added items to their cart and is ready to proceed to checkout.
Scenario	1) User selects the desired items in their cart and proceeds to checkout. 2) User selects a payment method (via Mobile or bank transfer). 3) User confirms the payment details and initiates payment. 4) System verifies the payment status and updates the order accordingly. 5) System will updates order status to "Paid".
Post Condition	Payment is successfully processed, and the order status is updated to "Paid".
Exception	If payment fails or encounters an error, user receives an error message.

3.2.3.10 Manage User Profile

Name	Manage User Profile
Actors	User
Description	User can manage their profile settings, including updating their password.
Precondition	User is logged into their account and navigates to the profile settings.
Scenario	1) User selects "Profile" or similar option from the menu. 2) User chooses the "Change Password" option. 3) User enters current password, new password, and confirms new password. 4) User clicks on the "Update Password" button.
Post Condition	User's password is successfully updated.

3.2.3.11 View Order History

Name	View Order History
Actors	User
Description	User can view their order history to see details of previous orders, including status and delivery information.
Precondition	User is logged into their account and navigates to the order history section.
Scenario	1) User selects "My Orders" or similar option from the menu. 2) System retrieves and displays a list of previous orders made by the user. 3) User selects a specific order to view detailed information. 4) System displays order details including items purchased, total cost, payment status, and delivery status.
Post Condition	User has successfully viewed the details of their order history.
Exception	If no orders are found for the user, a message indicating "No Orders Found" is displayed. User may be prompted to continue shopping or contact support for assistance.

3.2.3.12 Admin Sign In

Name	Admin Sign In
Actors	Admin
Description	Admin can sign in to access the admin dashboard and perform administrative tasks.
Precondition	Admin credentials are verified and only permitted admins have access to sign in.
Scenario	1) Admin navigates to the Sign In page. 2) Admin enters valid admin username and password. 3) Admin clicks on the "Sign In" button.
Post Condition	Admin successfully signs in and gains access to the admin dashboard.
Exception	If invalid credentials are entered, an error message is displayed. Admin may retry entering correct credentials to sign in.

3.2.3.13 Admin Dashboard Access

Name	Admin Dashboard Access
Actors	Admin
Description	Admin can access the admin dashboard to perform administrative tasks.
Precondition	Admin has successfully signed in to the system using valid admin credentials.
Scenario	1) After signing in, admin redirect to the dashboard page. 2) Admin is presented with an interface displaying various administrative options and functionalities to do.
Post Condition	Admin gains full access to the admin dashboard and can perform administrative tasks as required.
Exception	If the admin credentials are invalid or unauthorized, admin can not access the dashboard.

3.2.3.14 Add new product Item

Name	Add new product Item
Actors	Admin
Description	Admin can add a new product item to the system such as name, description, price, and image.
Precondition	Admin has successfully signed in to the system and accessed the admin dashboard.
Scenario	1) Admin navigates to the "New brand" section within the admin dashboard. 2) Admin fills out the required fields for the new product, including name, description, and price. 3) Admin uploads an image of the product and verifies all details are accurate. 4) Admin submits the new product information.
Post Condition	The new product item is added to the system database and becomes available for users to view and purchase.
Exception	If any required field is missing or invalid, the system prompts the admin to correct the information before proceeding.

3.2.4 Class Diagram

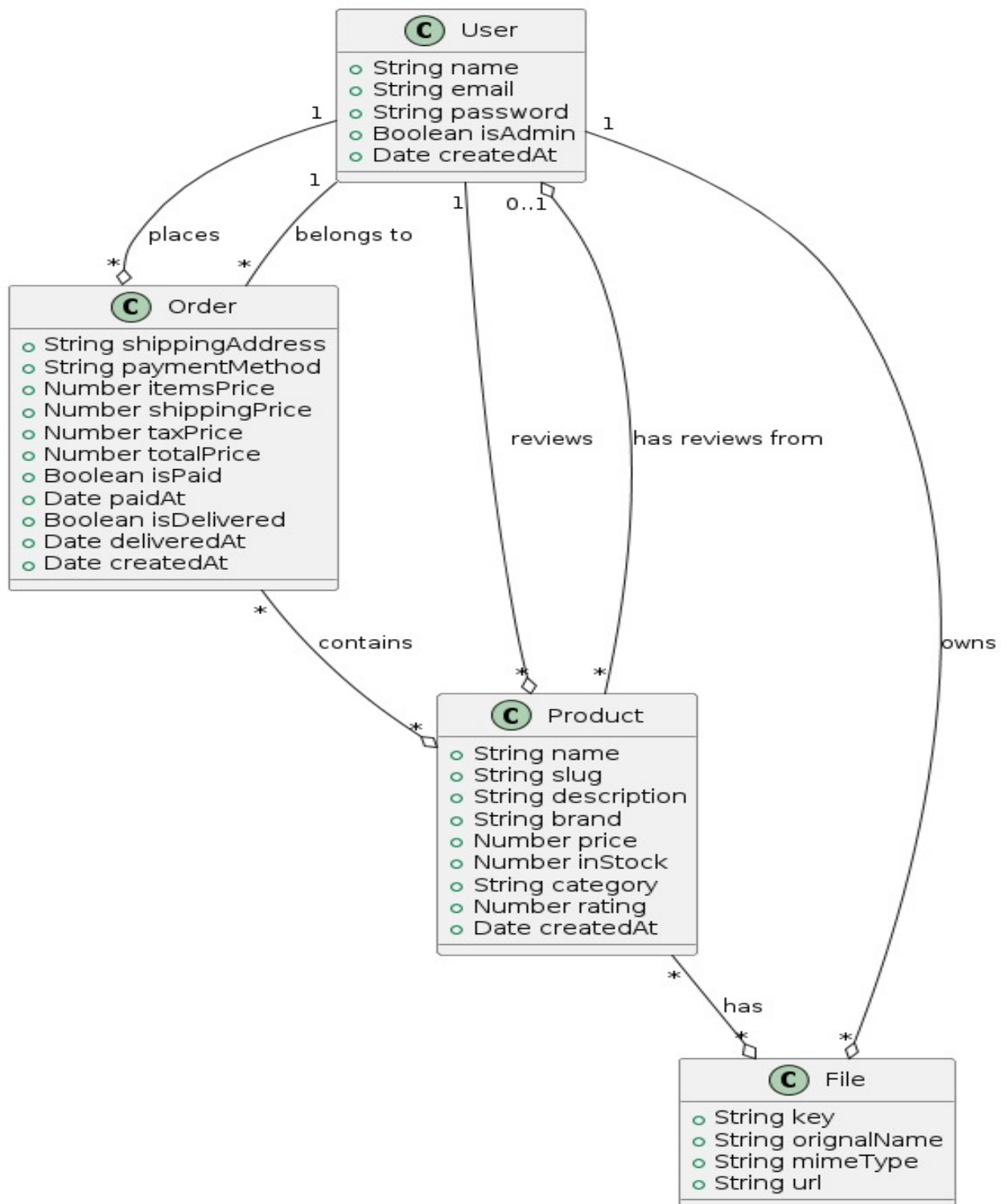


FIGURE 3.3: Class Diagram

3.3 Design Description

In our WeCare project, we are using the MVC (Model-View-Controller) design architecture model. This model divides the application into three interconnected components such as;

3.3.1 Model

- **Role:** Represents the data and business logic of the application.
- **Components:** Data schemas and models for products, users, orders, etc. It is typically implemented with MongoDB.

3.3.2 View

- **Role:** Represents the UI components that presents data and interact with the end user.
- **Components:** Front-end components developed using React.js, such as viewing home page, managing carts, and handling user profiles.

3.3.3 Controller

- **Role:** Handles the user input and interacts with the model to update the view.
- **Components:** Express.js routes and middleware that manage HTTP requests and responses, user authentication, cart management, and order processing.

Chapter 4

Implementation and Evaluation

4.1 Development Stages

4.1.1 Strategy

We employ an iterative model to ensure constant improvement and development of features in developing the WeCare project. This strategy allows us to break down the project into handy phases and simplifies regular adjustments.

Primarily, we conduct detailed planning and requirements gathering to define the project's scope and objectives. Each iteration includes detailed analysis and design, followed by incremental implementation of features. Rigorous testing at each stage guarantees the functionality and performance meet user potentials. Regular evaluations guide subsequent iterations that ensured the final product aligns thoroughly with the project goals.

4.2 System Integration

System integration in the WeCare project is a key step to ensure unified interaction between several components and modules. Using the iterative model, we integrate components slowly. We started with core functionalities such as user authentication, home page, and cart management.

The backend, developed using Node.js and Express.js that connects with the MongoDB database to manage data. The frontend, built with React.js to display dynamic content and handle user actions. Payment gateways like Mobile and bank transfer options are integrated to simplify the transactions.

4.2.1 User Interface

The User Interface (UI) of the WeCare project is designed to provide a friendly and intuitive experience for end users. UI highlights ease of navigation and accessibility built with React.js. It allows users to see the complete menu on the home page without requiring Sign in or registration. The interface includes features such as:

- Search bar for finding desired products
- Dynamic cart for managing orders
- User profile section for updating passwords

The admin panel is designed for well-organized management of products that allows admin to add new products or brands and other administrative tasks with ease. The UI also integrates secure payment options that offer users the choice of Mobile or bank transfer and history views. Each element of the interface is designed to improve usability and modernize the ordering process.

4.2.1.1 Home Screen

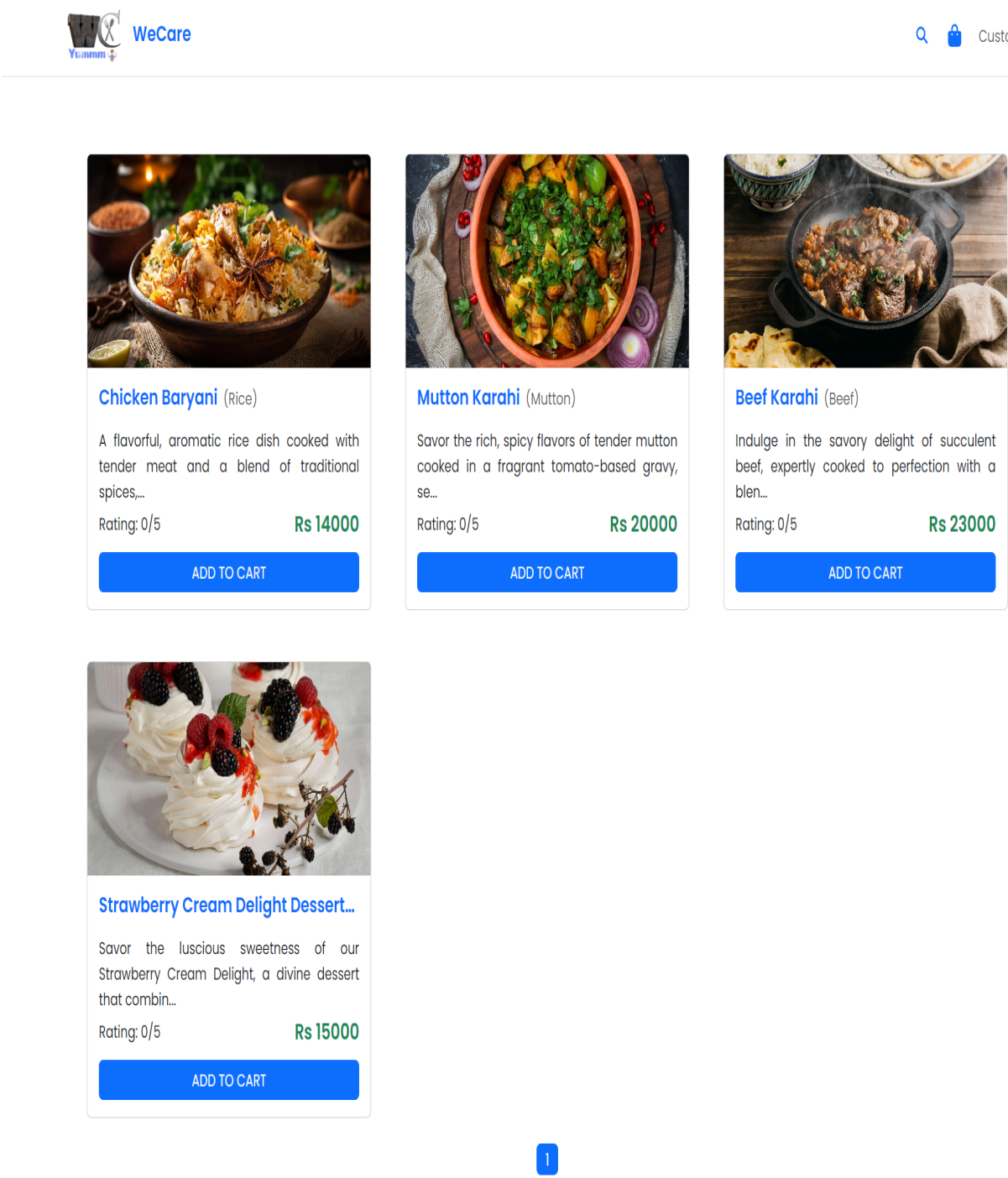


FIGURE 4.1: Home Screen

4.2.1.2 View Product Screen

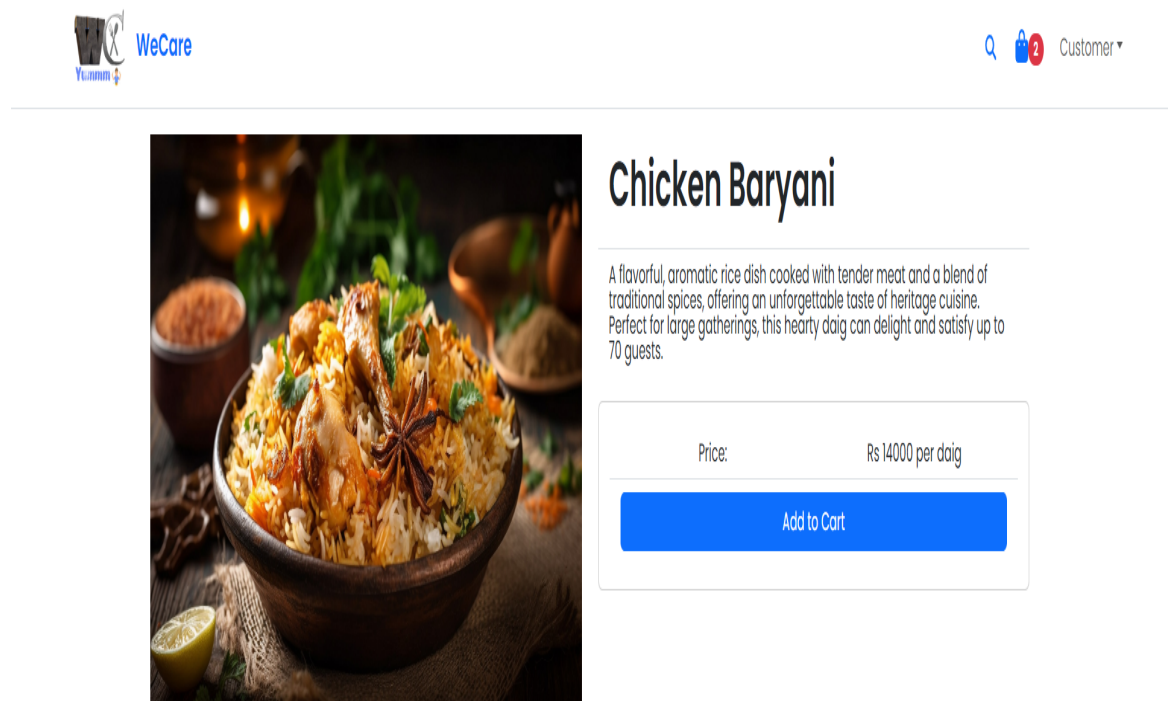


FIGURE 4.2: View Product Screen

4.2.1.3 Cart Screen

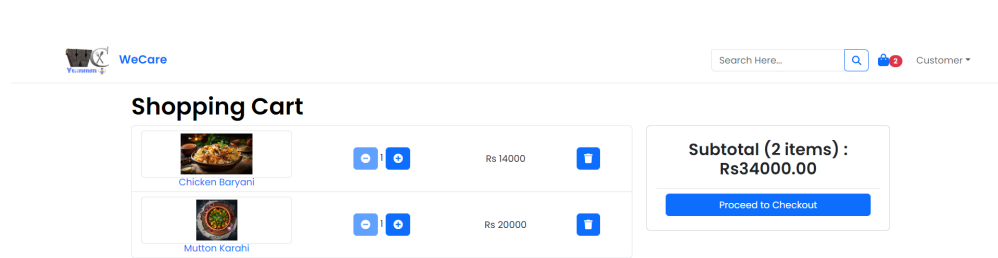


FIGURE 4.3: Cart Screen

4.2.1.4 Product Search

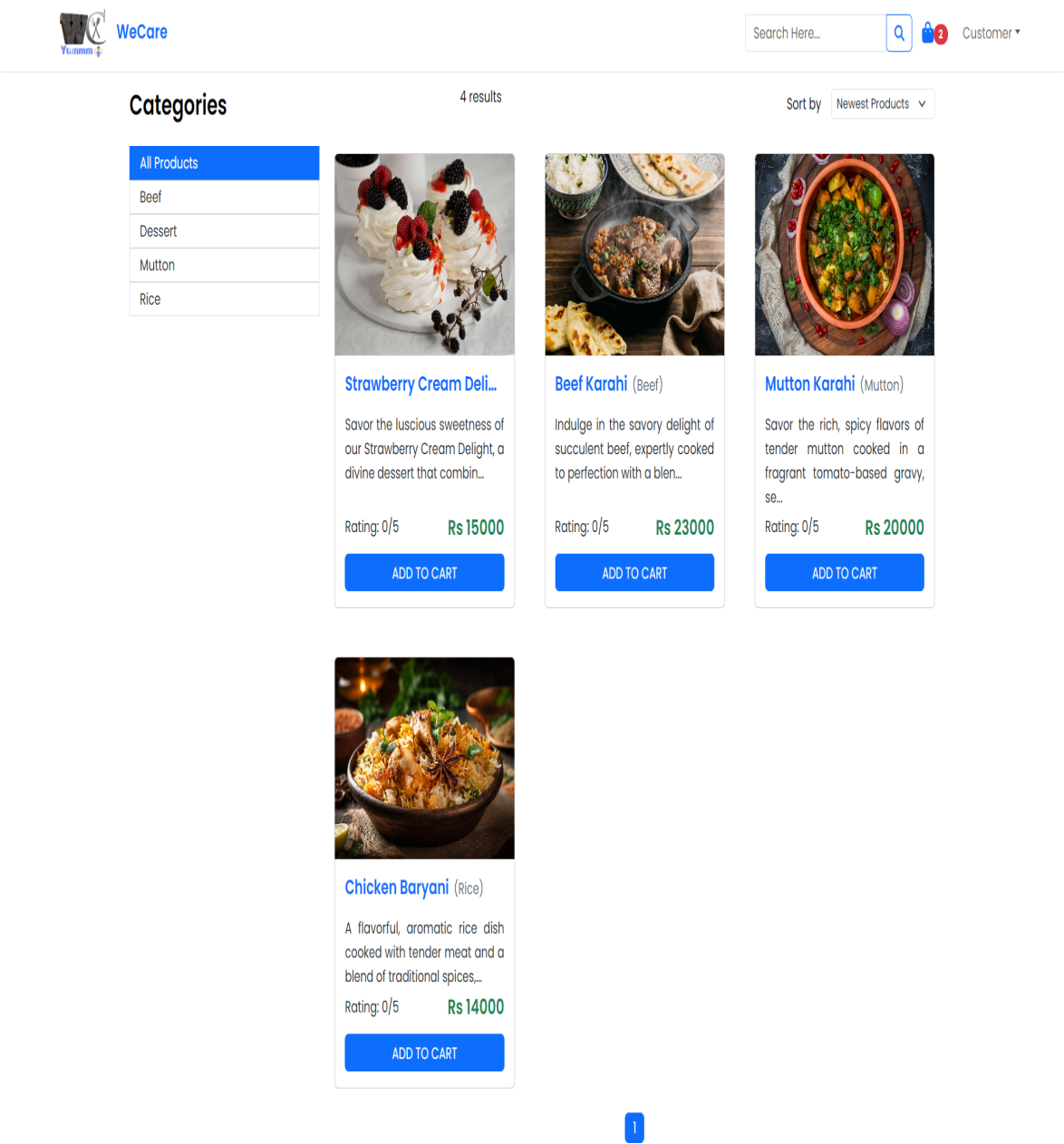


FIGURE 4.4: Product Search

4.2.1.5 Sign Up

 Sign in

Sign Up

Name

Email

Password

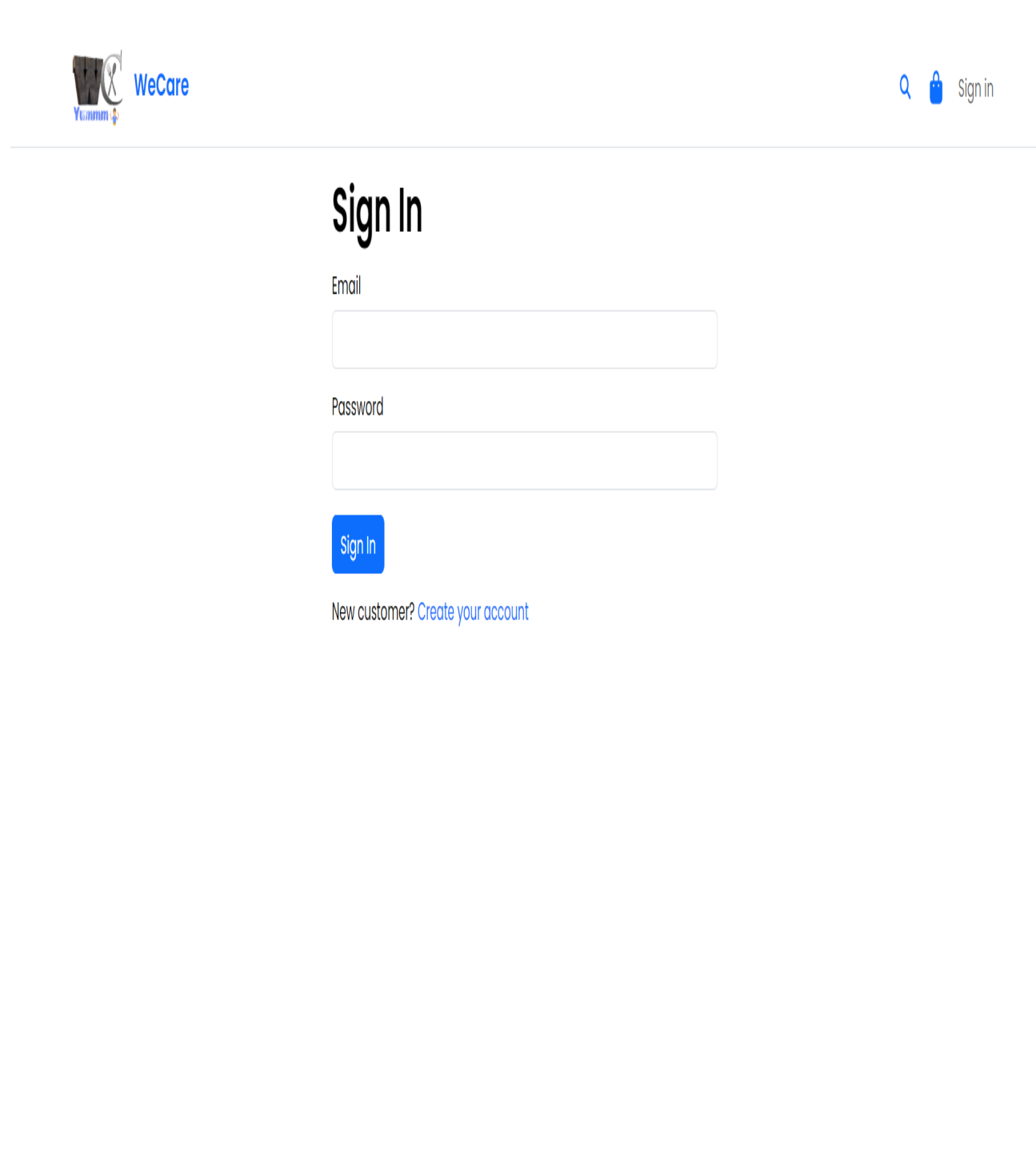
Confirm Password

[Sign Up](#)

Already have an account? [Sign-In](#)

FIGURE 4.5: Sign Up

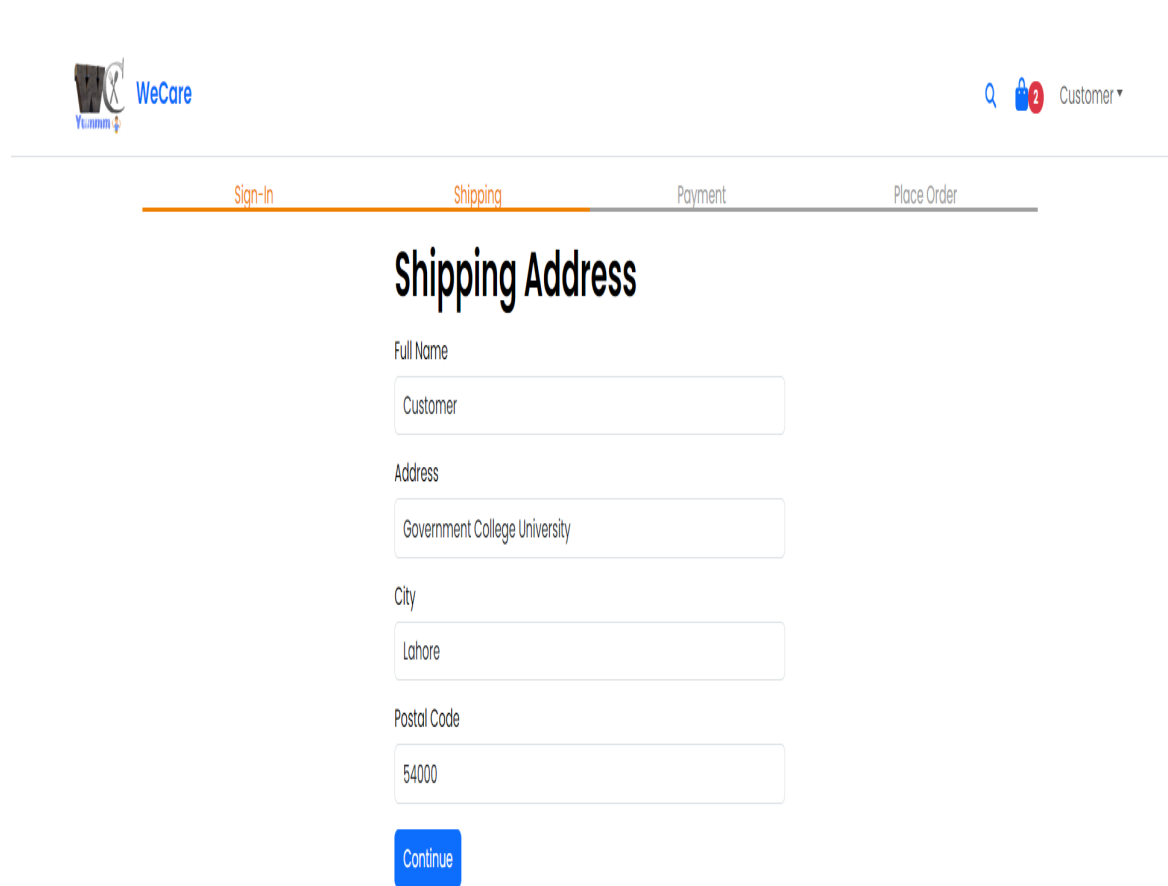
4.2.1.6 Sign In



The screenshot shows a web page for 'WeCare Yummy'. In the top left corner is the logo, which consists of a stylized 'W' and 'C' with a fork and knife, and the text 'WeCare Yummy' below it. In the top right corner, there is a magnifying glass icon, a blue shopping bag icon, and the text 'Sign in'. The main content area has a large heading 'Sign In'. Below this heading are two input fields: the first is labeled 'Email' and the second is labeled 'Password'. Below the password field is a blue button with the text 'Sign In'. At the bottom of the form, there is a link that says 'New customer? [Create your account](#)'.

FIGURE 4.6: Sign In

4.2.1.7 Shipping Address



The screenshot shows the 'Shipping Address' page of the WeCare system. At the top left is the WeCare logo. At the top right are icons for a search, a shopping cart with a '2' badge, and a 'Customer' dropdown menu. Below these is a progress bar with four steps: 'Sign-In', 'Shipping' (highlighted in orange), 'Payment', and 'Place Order'. The main heading 'Shipping Address' is centered. Below it are four input fields: 'Full Name' with the value 'Customer', 'Address' with 'Government College University', 'City' with 'Lahore', and 'Postal Code' with '54000'. A blue 'Continue' button is at the bottom.

WeCare

Search Cart 2 Customer ▾

Sign-In Shipping Payment Place Order

Shipping Address

Full Name

Address

City

Postal Code

Continue

FIGURE 4.7: Shipping Address

4.2.1.8 Payment Method Screen

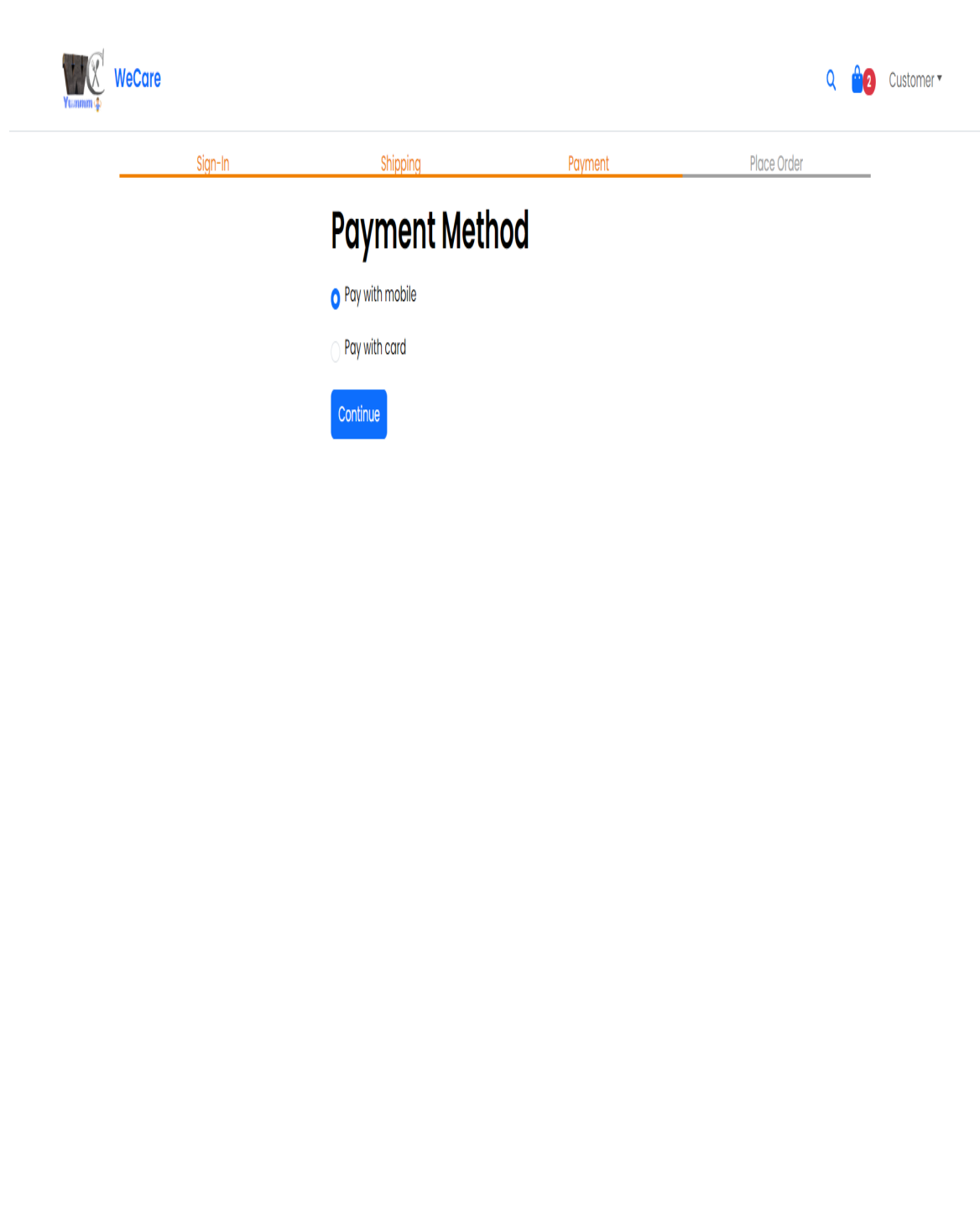


FIGURE 4.8: Payment Method Screen

4.2.1.9 Order Screen

 WeCare

 Customer ▾

Sign-In

Shipping

Payment

Place Order

Preview Order

Shipping

Name: Customer
Address: Government College University,Lahore, 54000,
[Edit](#)

Payment

Method: mobile
[Edit](#)

Items

	1	Rs14000
Chicken Biryani		
	1	Rs20000
Mutton Karahi		
Edit		

Order Summary

Items	Rs34000.00
Shipping	Rs0.00
Tax	Rs5100.00
Order Total	Rs39100.00

Place Order

FIGURE 4.9: Order Screen

4.2.1.10 Order History Screen

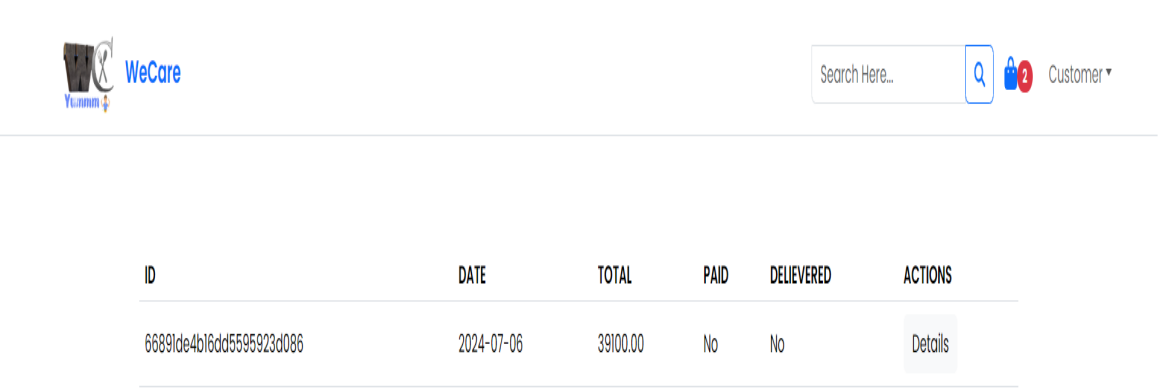


FIGURE 4.10: Order History Screen

4.2.1.11 Order Details Screen

 WeCare

Search Here...   2 Customer ▾

Order 66891de4b16dd5595923d086

Shipping

Name: Customer
Address: Government College University, Lahore, 54000,

Not Delivered

Payment

Method: mobile

Not Paid

Items

	Rs 14000
	Rs 20000

Order Summary

Items	Rs 34000.00
Shipping	Rs 0.00
Tax	Rs 5100.00
Order Total	Rs 39100.00

[Pay Now](#)

FIGURE 4.11: Order Details Screen

4.2.1.12 Payment Details Screen

Payment Details [X]

Phone Number
03123456789

CNIC (Last 6 digits)
345678

Pay Now

Order Summary

Items	Rs 34000.00
Shipping	Rs 0.00
Tax	Rs 5100.00
Order Total	Rs 39100.00

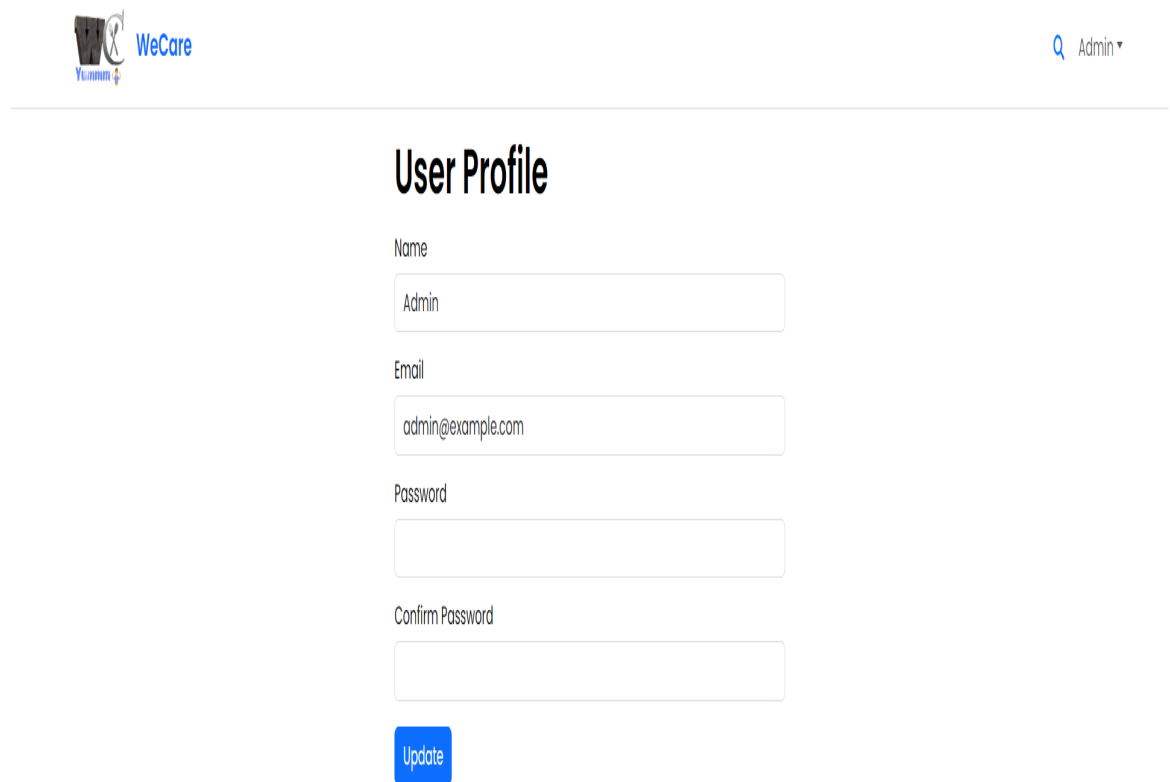
Pay Now

Payment
Method: mobile
Not Paid

Items

 Chicken Biryani	Rs 14000
 Mutton Karahi	Rs 20000

FIGURE 4.12: Payment Details Screen

4.2.1.13 User Profile Screen

The screenshot shows a web application interface for a 'User Profile' screen. At the top left, there is a logo with the text 'WeCare' and 'Yummi' below it. At the top right, there is a user profile icon and the text 'Admin'. The main heading is 'User Profile'. Below the heading, there are four input fields: 'Name' (containing 'Admin'), 'Email' (containing 'admin@example.com'), 'Password', and 'Confirm Password'. At the bottom, there is a blue 'Update' button.

WeCare
Yummi

Admin

User Profile

Name

Admin

Email

admin@example.com

Password

Confirm Password

Update

FIGURE 4.13: User Profile Screen

4.2.1.14 Customer Options

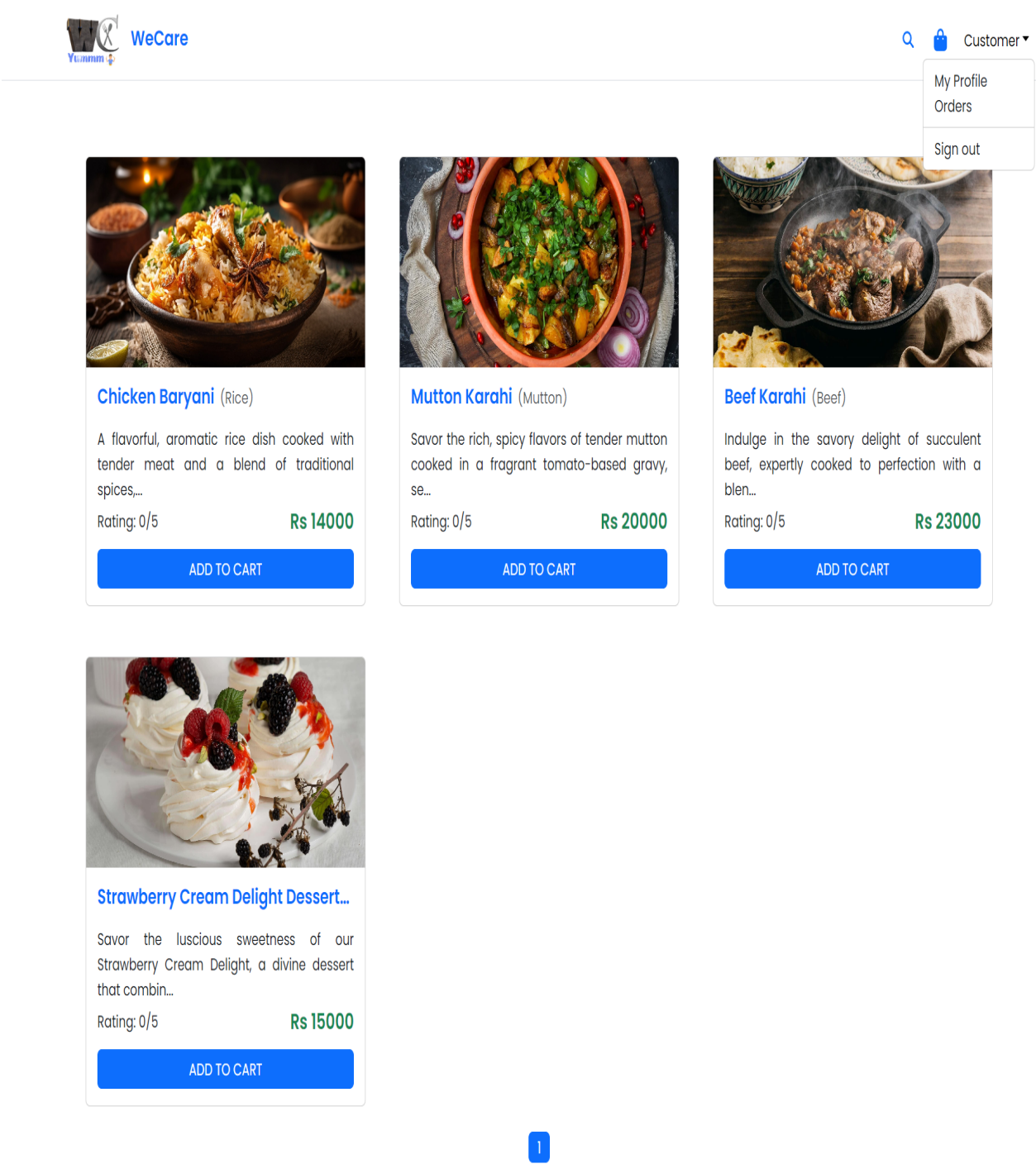


FIGURE 4.14: Customer Options

4.2.1.15 Admin Options

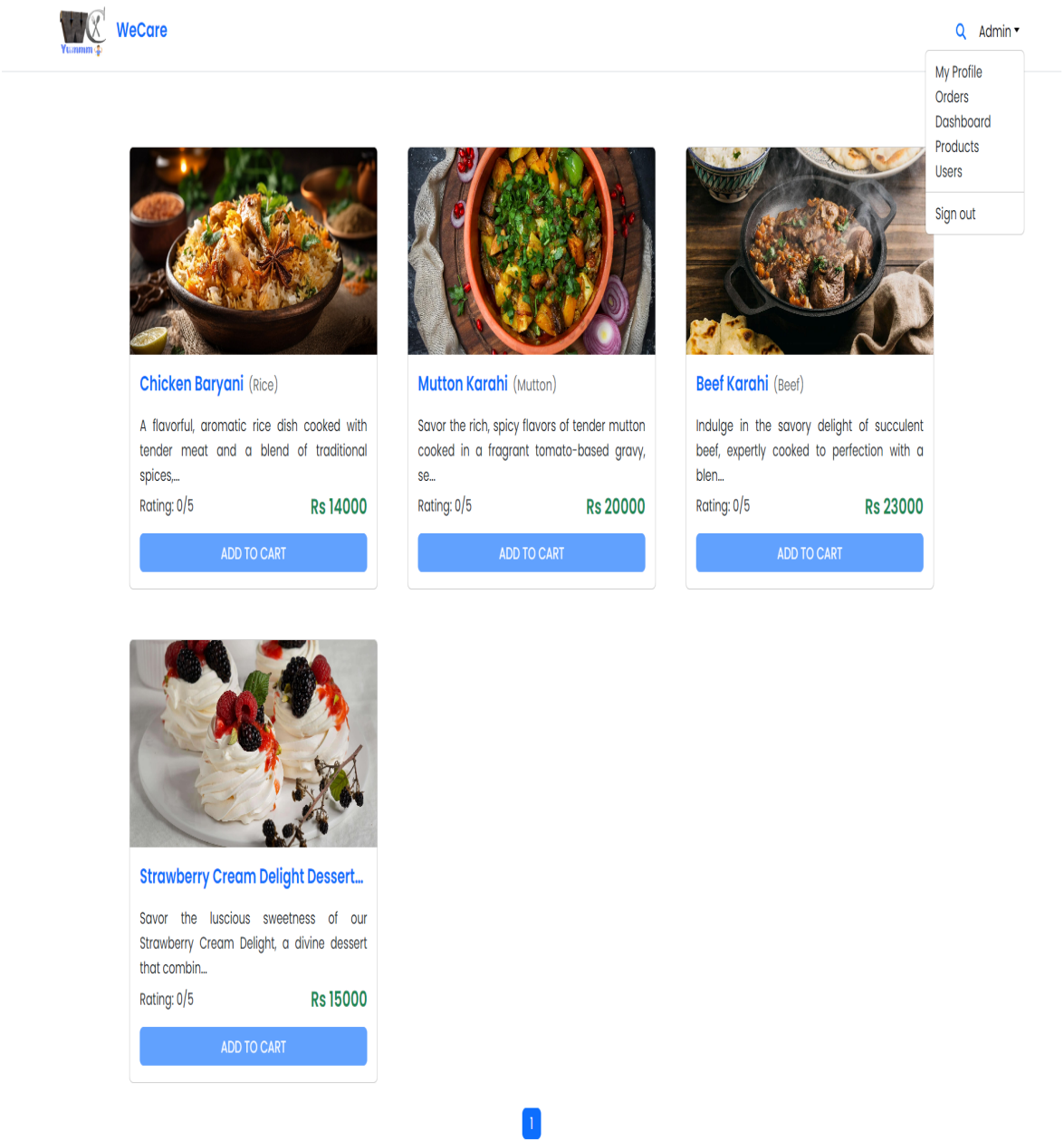


FIGURE 4.15: Admin Options

4.2.1.16 Add New Product Screen

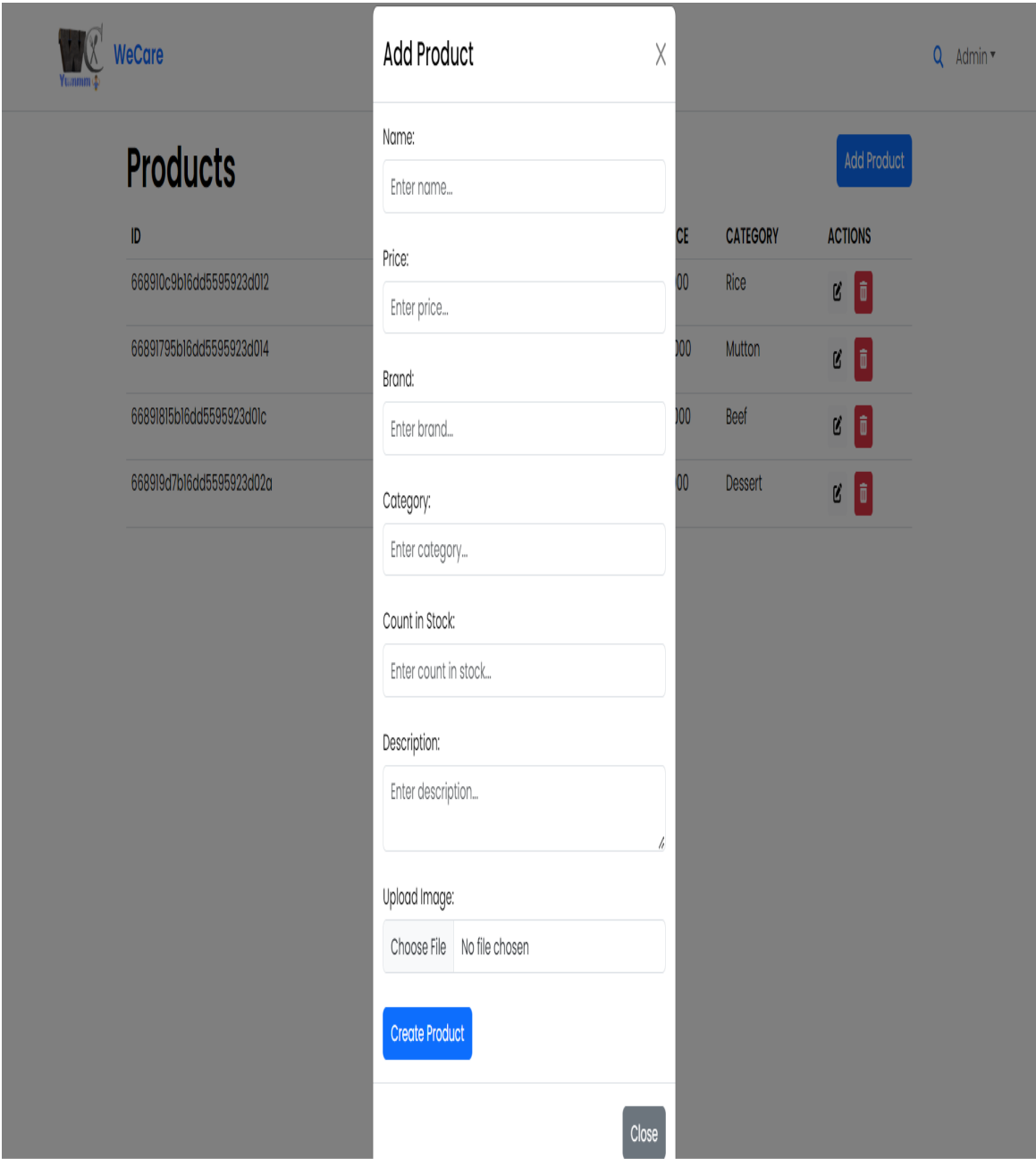


FIGURE 4.16: Add New Product Screen

4.2.1.17 Admin Order Management Screen

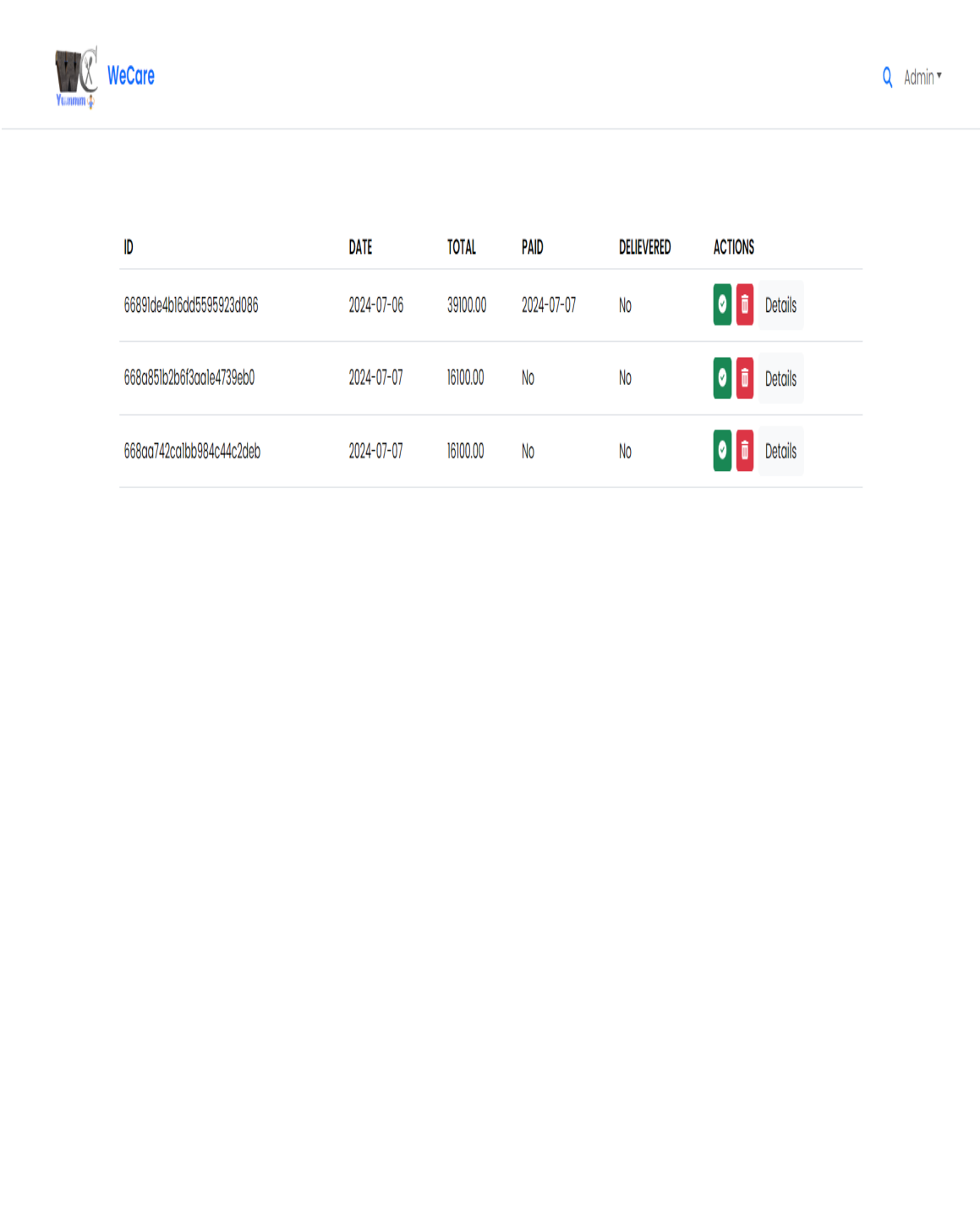


FIGURE 4.17: Admin Order Management Screen

4.2.1.18 Admin Product Management Screen

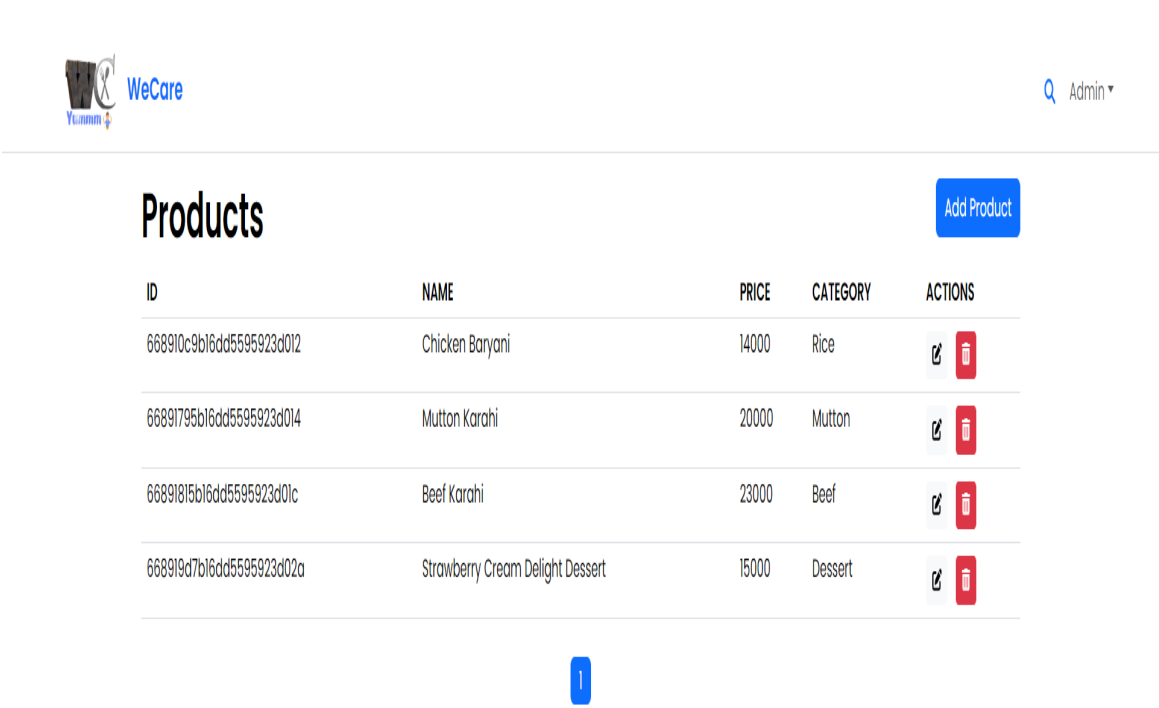


FIGURE 4.18: Admin Product Management Screen

4.2.1.19 Admin User Management Screen

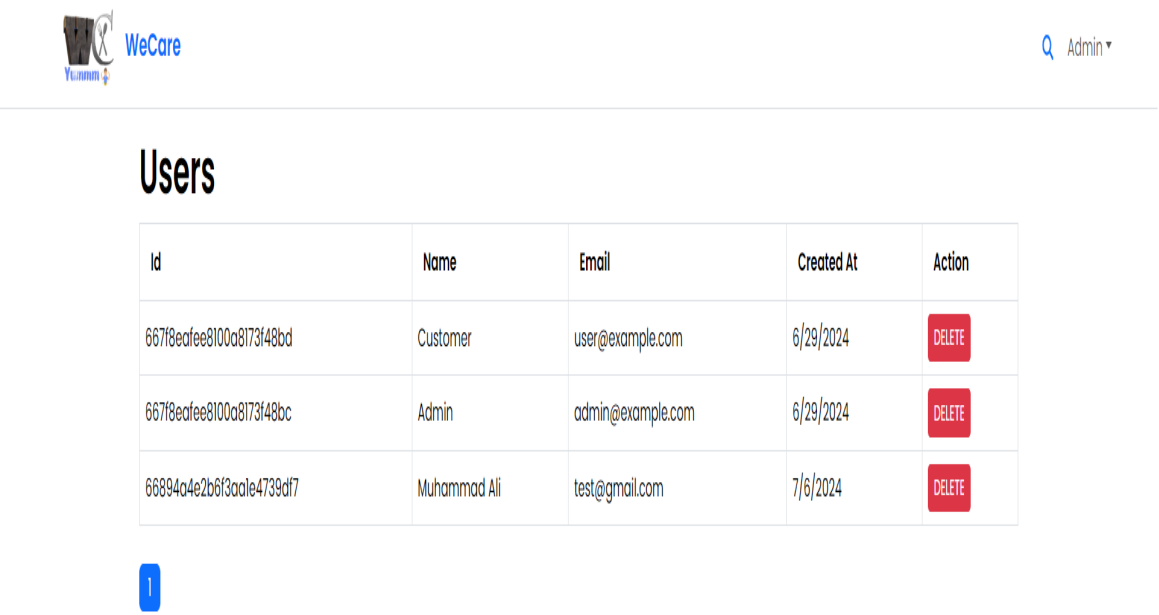
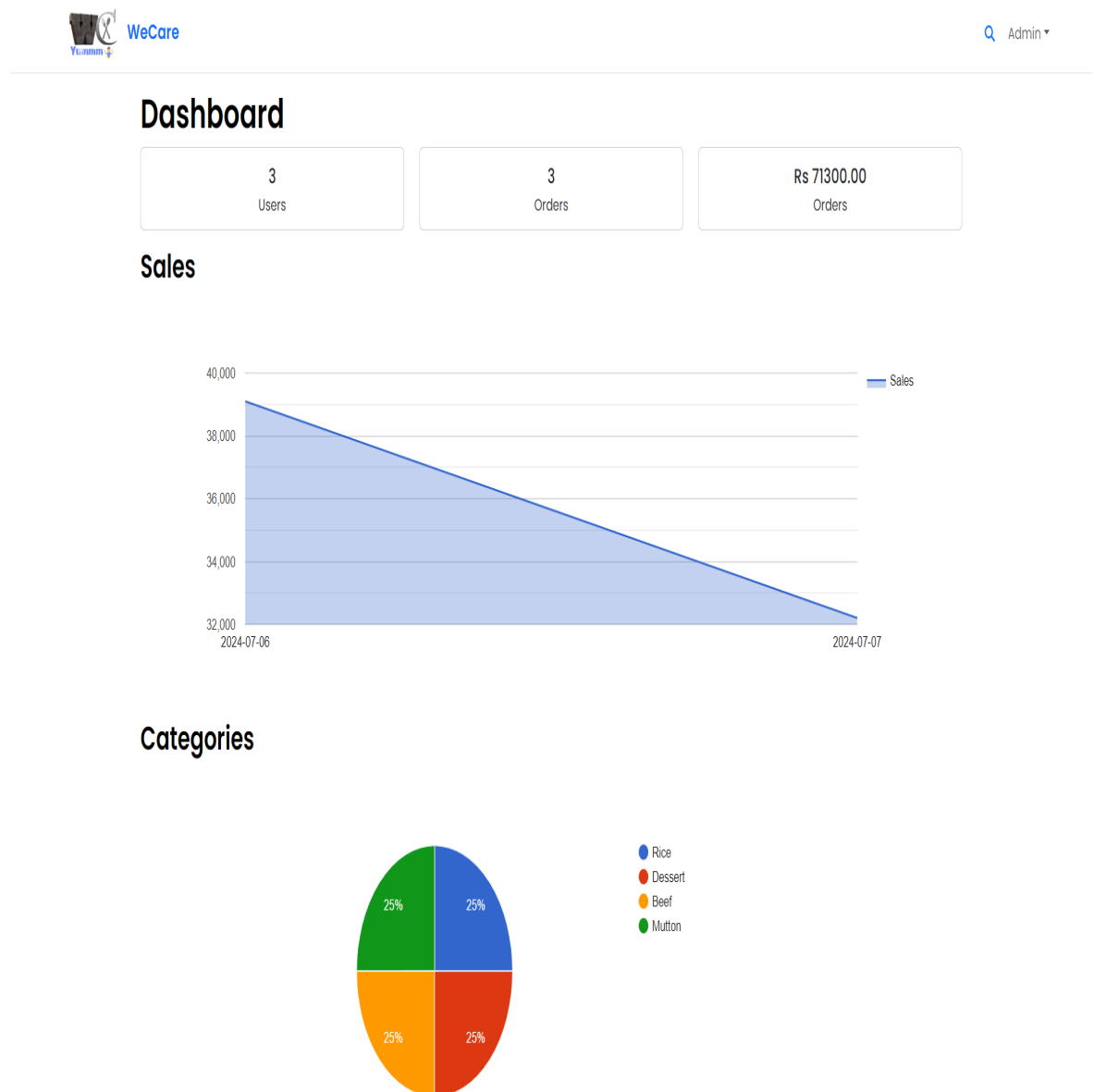


FIGURE 4.19: Admin User Management Screen

4.2.1.20 Admin Dashboard Screen



4.2.1.21 Edit Product Screen

 Admin ▾

Edit Product

668910c9b16dd5595923d012

Name

Slug

Price

Image File

Category

Brand

Count In Stock

Description

FIGURE 4.21: Edit Product Screen

4.3 Evaluation

Evaluation in the WeCare project includes evaluating each iteration against pre-defined goals and approved objectives. Evaluation also emphasizes on detecting areas for enhancement, whether in usability, performance, or feature development, and prioritizing these adjustments for upcoming iterations. Through constant evaluation, the project goals to refine user interactions, modernize administrative tasks, and enhance system reliability. This iterative approach guarantees that each phase of development contributes to a strong and user-centric catering order management system.

4.4 Unit Testing

Unit testing is an important aspect of software development within the WeCare project. It helps to validate individual components or units of code to ensure they function properly in isolation. In this context, unit testing includes creating and running automated tests on small such as functions, methods, or classes. These tests confirm that each unit behaves as expected and adhere to design specifications. By finding and fixing defects early in the development cycle, unit testing contributes to better code quality, and enhances the reliability and maintainability of the WeCare system.

4.5 Functional Testing

Functional testing in the WeCare project includes validating complete functionalities of the application to make sure that they meet stated requirements. It tests scenarios like user interactions, order processing, and admin operations to check they perform as planned. Functional testing guarantees that the whole system functions properly from end to end and enhance reliability and user satisfaction.

4.6 Test Cases

4.6.1 User Registration

Purpose	This test case is to verify the successful registration of a user by providing valid information and completing the registration process.
Preconditions	The registration form is displayed and ready to accept user input.
Test Data	- Full Name: John Smith - Email Address: muhammadali@gmail.com - Password: password123
Steps	1. Enter the full name as "Muhammad Ali" in the corresponding field. 2. Enter the email address as muhammadali@gmail.com in the corresponding field. 3. Enter the password as password123 in the corresponding field. 4. Click on the "Register" button. 5. Wait for the registration process to complete.
Expected Result	1. The system should accept the provided information and register the user successfully. 2. A confirmation message should be displayed, indicating that the registration was successful.
Actual Status	The user registration was successful.
Status	Pass

4.6.2 User Sign In

Purpose	This test case is to verify the successful sign in of a registered user using correct credentials.
Preconditions	The sign in form is displayed and ready to accept user input.
Test Data	- Email Address: test@example.com - Password: password123
Steps	1. Enter the email address as test@example.com in the corresponding field. 2. Enter the password as password123 in the corresponding field. 3. Click on the "Sign In" button.
Expected Result	1. The system should authenticate the user with the provided credentials. 2. Upon successful sign in, the user should be redirected to the home page.
Actual Status	The user was able to sign in successfully.
Status	Pass

4.6.3 Sign Out

Purpose	To verify that users can successfully sign out from their account.
Preconditions	User is signed in to the WeCare system.
Test Data	None
Steps	1. Click on the user profile. 2. Locate and click on the "Sign Out" button.
Expected Result	The system should log the user out and redirect them to the homepage.
Actual Status	The user was successfully logged out.
Status	Pass

4.6.4 Add Product to Cart

Purpose	To verify that users can add products to their cart successfully.
Preconditions	User is Signed in in and viewing the product details page.
Test Data	- Product: Name: "", Price: "", Description: ""
Steps	1. Go to the home page. 2. If you want to see details, click on the "View more" button. 3. Click on the "Add to cart" button
Expected Result	The product should be added to the cart with quantity 1.
Actual Status	The product was successfully added to the cart.
Status	Pass

4.6.5 Remove Product from Cart

Purpose	To verify that users can remove products from their cart successfully.
Preconditions	Cart must have some products.
Test Data	- Product: Name: "", Price: "", Quantity: ""
Steps	1. Click to the cart button where products are listed. 2. Locate the product added product in the cart. 3. Click on the "delete" icon next to the product.
Expected Result	The product should be successfully removed from the cart.
Actual Status	The product was successfully removed from the cart.
Status	Pass

4.6.6 Search Products

Purpose	To verify that users can search for desired products using the search bar.
Preconditions	User is on the homepage where the search bar is accessible.
Test Data	- Search Query: "Product name"
Steps	1. Locate the search bar on the webpage. 2. Write the search query such as Product name into the search bar. 3. Click on the search icon.
Expected Result	Products matching the search query "Product name" should be displayed in the search results.
Actual Status	Products matching the search query "Product name" were displayed.
Status	Pass

4.6.7 Payment Method

Purpose	To verify that users can select and proceed with a payment method during checkout.
Preconditions	User is on the checkout page with available payment options displayed.
Test Data	- Selected Payment Method: Mobile
Steps	1. click on the "Proceed to checkout". 2. Select "Mobile or Bank transfer" as the preferred payment method. 3. Complete any required fields or authentication steps for payment method. 4. Click on the "Pay order" or equivalent button.
Expected Result	The system should successfully process the payment through Mobile or Bank transfer and confirm the order.
Actual Status	Payment through Mobile or Bank transfer was successfully processed.
Status	Pass

4.6.8 Manage User Profile

Purpose	To verify that users can update their password in "My profile" and view order history in "My orders".
Preconditions	User is Signed into the WeCare system and clicks on the Profile name.
Test Data	- Password: newPassword456 - Confirm Password: newPassword456
Steps	1. Click to the "Profile" after signing in. 2. Enter the password and the confirm same password. 3. Click on the "Update" button to update. 5. Navigate to the "My Orders". 6. Click on "My Orders" to view the order history.
Expected Result	1. The system should update the password. 2. The user should be able to view a list of previous orders with details such as order status, items ordered, and dates.
Actual Status	Password was successfully updated. Order history was displayed correctly.
Status	Pass

4.6.9 Admin Sign In Access

Purpose	To verify that only authorized admin can sign in and access the admin dashboard.
Preconditions	Admin credentials (username and password) are known.
Test Data	- Username: admin@example.com - Password: adminPassword123
Steps	1. Enter the admin username and password in the respective fields. 2. Click on the "Sign In".
Expected Result	The system should authenticate the admin credentials. Upon successful sign in, the admin should be redirected to the admin dashboard.
Actual Status	Admin credentials were authenticated successfully.
Status	Pass

4.6.10 View Menu Display

Purpose	To verify that the menu displays correctly on home page to users without requiring sign in.
Preconditions	The user redirects to the home page of the WeCare website.
Steps	1. Open the WeCare website and see menu on the home page. 2. Verify that all products are displayed with their names, prices, and images.
Expected Result	All products from the menu should be visible to the user.
Actual Status	The menu page displayed all products correctly.
Status	Pass

4.6.11 Admin Dashboard Management

Purpose	To verify that the admin can access and manage the dashboard functionalities successfully.
Preconditions	Admin credentials are authenticated and the admin is signed in to the WeCare system.
Test Data	None
Steps	1. Sign in to the using admin account with valid credentials. 2. Navigate to the admin dashboard section. 3. Verify that the dashboard displays options to manage administrative tasks.
Expected Result	The admin should be able to access all dashboard functionalities without errors.
Actual Status	All dashboard functionalities were accessible and operational.
Status	Pass

Chapter 5

Conclusion & Future Work

5.1 Conclusion

The WeCare: Online Catering Reservation System project signifies a complete solution for managing catering orders for events that ensures a unified and user-friendly experience for both customers and admins. By using the modern technologies such as Node.js, Express.js, MongoDB, and React.js, WeCare delivers a well-organized and scalable platform that streamlines the process of viewing menu, placing orders, managing user profiles, and handling payments. The iterative development model has allowed constant improvement and refinement of features. It ensures that the system meets the developing needs of its users. With strong functionalities for both customers and admins, WeCare is crafted to improve the catering order management process.

5.2 Future Work

Future work for WeCare aims to enhance system functionality and user experience by developing a mobile app for iOS and Android, integrating advanced analytics for better decision-making, and adding more payment options, including digital wallets and cryptocurrency. Plans also include AI-powered recommendations, real-time order tracking, multilingual support, a comprehensive customer feedback system, and integration with event management tools. These enhancements will provide users with greater accessibility, personalized experiences, and streamlined event planning, ultimately improving overall customer satisfaction and operational efficiency.

Bibliography

- [1] Rainer Alt. Digital transformation in the restaurant industry: Current developments and implications.
https://www.researchgate.net/publication/351599584_Digital_Transformation_in_the_Restaurant_Industry

Index

Computer Science, [i](#)